StockSense AI

Technical Specification, Machine Learning Architecture & Data Flow Guide

Project Overview

StockSense AI is an end-to-end financial intelligence and price forecasting system. It connects a reactive, type-safe **React 19 + TypeScript + Vite** dashboard to an asynchronous **FastAPI** Python service. The system downloads historical global market bars, transforms them into non-linear, scale-free technical indicators, and fits an ensemble **RandomForestRegressor** using a strictly chronological hold-out window. Predicted trajectories are benchmarked against a naive persistence baseline and recursively projected forward for 1, 7, and 30 trading sessions.

1. Data Acquisition & Ingestion: How Market Data Is Fetched

Market data is streamed dynamically from **Yahoo Finance** via the Python yfinance library within backend/app/market.py. The application is completely dynamic and does not rely on static CSV files or hardcoded prices.

- **Ticker Symbol Validation:** All incoming queries are validated against the regex ^[A-Z0-9][A-Z0-9.\-^=]{0,14}$. Malformed or SQL-injection style inputs are rejected immediately with HTTP 400 before reaching the network layer.
- **Global & Indian Market Coverage:** The system seamlessly supports US exchanges (NASDAQ, NYSE) and Indian equities via National Stock Exchange suffixes (e.g., RELIANCE.NS, TCS.NS, INFY.NS). The frontend automatically binds appropriate currency symbols ($, INR).
- **Historical Lookback Horizons:** Requests pull daily bars across 5 allowable periods: 1mo, 3mo, 6mo, 1y, and 5y. For machine learning training, a 5-year lookback (~1,250 trading days) is loaded to guarantee adequate samples for moving average warm-up and testing.
- **Data Hygiene & Sanitization:** Raw quotes are cleaned: duplicate timestamps are dropped keeping the latest session, non-trading weekend gaps are parsed natively, rows missing Close prices are dropped, and invalid negative/infinite prices are filtered out.
- **Error Mapping Without Stack Traces:** Upstream timeouts, delisted symbols, and Yahoo rate limits are intercepted and mapped into client-safe exceptions (TickerNotFound → 404, RateLimited → 429, UpstreamUnavailable → 502), ensuring internal server internals are never exposed.

2. Storage Architecture: Why No Database (Supabase / SQL) Is Used

A common question during technical review is why external databases like Supabase, PostgreSQL, or MongoDB were intentionally omitted from the architecture. The design uses lightweight, high-performance local persistence tailored for machine learning inference:

Storage Layer	Implementation	Technical Purpose & Advantages
In-Memory Quote Cache	`market.py` `Thread-safe dictionary with monotonic clock TTL`	Caches live quotes for 300 seconds (5 min). Prevents Yahoo Finance rate-limiting (HTTP 429) during repeated user clicks. Invalid tickers (404) are cached for 60s.
Model Persistence	`joblib.dump()` `backend/models/*.joblib`	Trained Random Forest estimators and evaluation metadata are serialized as binary files on disk. Subsequent predictions load from disk in <10ms, eliminating redundant retraining.
Why No SQL / Supabase?	`Stateless REST Design`	1. Market data is already hosted and indexed globally by Yahoo Finance; duplicating price tables in SQL adds latency and maintenance overhead. 2. There are no user accounts, passwords, or transactional states to store. 3. Loading numpy matrices from Joblib binaries is significantly faster than parsing rows from an external database over network sockets.

3. Feature Engineering: Formulae & Zero Look-Ahead Bias

All features are implemented in backend/app/ml/features.py. A fundamental constraint is that for any session *t*, **only prices and volumes occurring at or before session *t* can be used**. Future data never leaks into features.

Feature Name	Mathematical Definition	Economic / Technical Purpose
`close_lag_k_rel`	$(\text{Close}_{t-k} / \text{Close}_{t}) - 1$ for $k \in \{1, 2, 3, 5, 10\}$	Captures short-to-medium momentum and price distance from prior sessions without unit dependence.

`ret_1 & ret_lag_k`	$ret_t = (Close_t / Close_{t-1}) - 1$ Lagged: $k \in \{1, 2, 3, 5\}$	Measures immediate velocity and autocorrelation of daily percentage price changes.
`dist_sma_w`	$(Close_t / SMA_w(t)) - 1$ for $w \in \{5, 10, 20, 50\}$	Quantifies trend extension: distance of current close from trailing 1-week, 2-week, 1-month, and 10-week moving averages.
`volatility_w`	$std(ret_{t-w..t})$ for $w \in \{5, 10, 20\}$	Captures market turbulence and regime shifts across 5, 10, and 20 trading sessions.
`range_pct`	$(High_t - Low_t) / Close_t$	Measures session price dispersion and intraday volatility.
`volume_chg`	$clip((Vol_t / Vol_{t-1}) - 1, -0.9, 3.0)$	Identifies institutional accumulation or distribution spikes, clipped to tame non-stationary anomalies.

Target Alignment (Shift -1): *The target variable is the next session's percentage return: $Target_t = (Close_{t+1} / Close_t) - 1$. Because 50-day Simple Moving Average requires 50 historical closes, the earliest 49 warm-up rows are discarded. The very last session has no target and is held out to seed future forecasts.*

4. Machine Learning Implementation: Random Forest Regressor

The algorithm is implemented in backend/app/ml/model.py using scikit-learn.

- **Why Random Forest?** Stock price dynamics contain non-linear interactions (e.g. crossing a moving average while volatility spikes). Linear models fail to capture these without manual combinatorial features. Recurrent networks (LSTMs) frequently overfit noisy daily stock data. Random Forest uses bootstrap aggregation (bagging) to average 150 diverse trees, dramatically curtailing variance without inflating bias.
- **Stationary Relative Returns:** Decision trees partition feature space into piecewise constant regions and *cannot extrapolate outside their training range*. Training on raw prices would artificially cap predictions at all-time highs. Training on scale-free returns (change / price) resolves this fundamental limitation.
- **Chronological 80/20 Train-Test Split:** Time-series observations are never randomly shuffled. Shuffling leaks future price information into training. The earlier 80% forms the training window; the latest 20% forms the untouched test set.
- **Naive Baseline Benchmarking:** In financial economics, asset prices approximate a martingale ($E[P_{t+1} \mid P_t] \approx P_t$). StockSense benchmarks Random Forest against the naive persistence baseline (predicted $P_{t+1} = P_t$). If the Random Forest fails to beat the baseline MAE, it reports that result honestly.

5. Recursive Multi-Step Forecasting Engine

In backend/app/ml/forecasting.py, multi-step forecasts (1, 7, and 30 trading sessions) are generated recursively using the one-step-ahead trained model:

1. **Step Execution:** At step *k*, compute technical features from the working history and predict relative price return.
2. **Dynamic History Update:** Compute predicted close $P_{t+k} = P_{t+k-1} \times (1 + \text{return})$ and append the predicted price to the working close series.
3. **Recalculate Trailing Windows:** Recompute lags, moving averages, and rolling volatilities incorporating the newly predicted close. Features are never held static.
4. **Transparent Unobservable Assumptions:** Because future High, Low, and Volume cannot be observed, the model holds future high-low range at the recent 20-session average and assumes volume change is 0. Future dates skip Saturdays and Sundays.
5. **Error Compounding:** 30-day forecasts are inherently less certain than 1-day forecasts because step 30 relies on 29 successive synthetic inputs. This limitation is explicitly communicated in the UI.

6. Master Parameter Reference Table

Category	Parameter Name	Default Value	Architectural & Mathematical Role
Model Hyperparameters	`n_estimators`	150	Number of independent decision trees in the ensemble. Provides variance reduction through bagging.
Model Hyperparameters	`max_depth`	10	Maximum depth of each tree. Prevents trees from memorizing localized market noise.
Model Hyperparameters	`min_samples_leaf`	2	Minimum samples required in terminal leaf nodes, preventing single-outlier splits.
Model Hyperparameters	`random_state`	42	Guarantees deterministic, reproducible tree splits and bagging bootstrap samples.
Model Hyperparameters	`n_jobs`	-1	Utilizes all available CPU cores for parallel tree fitting.
Dataset & Split	`TRAIN_FRACTION`	0.80 (80%)	Chronological partition ratio: earliest 80% train, subsequent 20% held-out test.
Dataset & Split	`MIN_TRAIN_SAMPLES`	60 sessions	Minimum required training rows; throws HTTP 422 if history is insufficient.
Dataset & Split	`MIN_TEST_SAMPLES`	20 sessions	Minimum hold-out test rows to ensure evaluation metric validity.
Technical Windows	`LAG_PERIODS`	1, 2, 3, 5, 10	Trading day intervals for lagged relative close features.
Technical Windows	`SMA_WINDOWS`	5, 10, 20, 50	Trailing sessions for Simple Moving Average distances.
Technical Windows	`VOL_WINDOWS`	5, 10, 20	Rolling window sizes for standard deviation of percentage returns.
Runtime Config	`CACHE_TTL_SECONDS`	300 (5 min)	In-memory cache lifetime for successful Yahoo Finance queries.
Runtime Config	`UPSTREAM_TIMEOUT`	10 seconds	Timeout limit for external Yahoo Finance HTTP requests before raising HTTP 504.
Runtime Config	`PERSIST_MODELS`	true	Enables automatic serialization of trained models to backend/models/.
API Routing	`horizon`	1 to 30	Allowable forward forecast sessions validated via FastAPI Query/Body constraints.

Academic Disclaimer: *Predictions generated by StockSense AI are statistical projections based on historical price patterns and technical market indicators. They are intended strictly for academic demonstration and research in computational finance and do not constitute financial advice.*